

# Números, operadores y precisión en JavaScript

*IEEE 754, BigInt, operadores aritméticos, lógicos, de asignación y bits, spread/rest y el objeto Math*

| SEMANA | SESIONES | DURACIÓN | MODALIDAD  |
|--------|----------|----------|------------|
| 02     | 03 y 04  | 4 horas  | Presencial |

CASO PRÁCTICO

## Cotizador comercial inteligente

Construirás una aplicación que calcula subtotal, descuento, IGV, envío y total; evita errores frecuentes de precisión y muestra cómo usar Number, BigInt, operadores, banderas de bits y spread/rest con una finalidad concreta.

| Curso               | Código     | Unidad               | Periodo               |
|---------------------|------------|----------------------|-----------------------|
| JavaScript Avanzado | 100000SI56 | Unidad 1: JavaScript | 2026 - Ciclo 2 Agosto |

## Ruta del laboratorio

La sesión combina explicación conceptual, experimentos controlados y construcción guiada. Ejecuta cada bloque en orden: los hallazgos sobre precisión justifican las decisiones del proyecto final.

| Bloque | Actividad                                        | Tiempo | Evidencia                     |
|--------|--------------------------------------------------|--------|-------------------------------|
| A      | Preparar el entorno y recuperar el proyecto base | 15 min | Carpetas y consola operativas |
| B      | Explorar Number, IEEE 754 y precisión            | 30 min | Resultados comentados         |
| C      | Aplicar operadores y precedencia                 | 40 min | Ejercicios verificados        |
| D      | Practicar bits, spread y rest                    | 25 min | Mini casos resueltos          |
| E      | Construir la estructura HTML                     | 30 min | Interfaz semántica            |
| F      | Aplicar estilos CSS                              | 25 min | Diseño responsivo             |
| G      | Programar y explicar la lógica JavaScript        | 55 min | Cotizador funcional           |
| H      | Probar, depurar y resolver el reto               | 20 min | Matriz de pruebas y mejora    |

*Tiempo total estimado: 240 minutos (4 horas).*

## Ficha del laboratorio

| Elemento           | Descripción                                                                                             |
|--------------------|---------------------------------------------------------------------------------------------------------|
| Logro de la semana | Resuelve cálculos web aplicando números, precisión, operadores, BigInt, bits, spread/rest y Math.       |
| Producto           | Aplicación web Cotizador comercial inteligente organizada en archivos HTML, CSS y JavaScript separados. |
| Estrategia         | Experimentación breve, explicación guiada, construcción incremental, pruebas y reto de extensión.       |
| Trabajo            | Individual durante la implementación; contraste de resultados en parejas al finalizar.                  |
| Requisitos         | Computadora, Visual Studio Code estable y navegador moderno con herramientas de desarrollo.             |

## Resultados de aprendizaje observables

- **Explica** por qué  $0.1 + 0.2$  no produce exactamente 0.3 en Number.
- **Diferencia** Number de BigInt y evita mezclarlos en una misma operación aritmética.
- **Selecciona** operadores según su propósito, sin depender de coerciones implícitas.
- **Aplica** operadores de bits para representar opciones y distingue spread de rest.
- **Implementa** cálculos monetarios con enteros en céntimos, validación y redondeo explícito.

- **Verifica** resultados mediante casos normales, límites y entradas inválidas.

## Convenciones y normas de trabajo

- **Escribe** el código exactamente en el archivo indicado; no mezcles HTML, CSS y JavaScript.
- **Guarda** los archivos antes de recargar el navegador y revisa la consola después de cada cambio.
- **Usa** igualdad estricta (`===` y `!==`) y conversiones explícitas con `Number()` cuando corresponda.
- **No copies** todos los bloques de una vez: prueba cada etapa y registra qué observas.

**Punto de partida:** La semana 1 introdujo variables, funciones, control de flujo, errores, DOM y eventos. Esta guía reutiliza esos fundamentos y concentra la atención en números y operadores.

## 1. Dos tipos numéricos: Number y BigInt

ECMAScript dispone de dos tipos numéricos incorporados. `Number` representa enteros y decimales mediante punto flotante binario de doble precisión; `BigInt` representa enteros de magnitud arbitraria. No son intercambiables: cada uno responde a un problema distinto.

| Criterio    | Number                                              | BigInt                                           |
|-------------|-----------------------------------------------------|--------------------------------------------------|
| Ejemplo     | 125.75                                              | 9007199254740993n                                |
| Representa  | Enteros y fracciones                                | Solo enteros                                     |
| Precisión   | Aproximada para muchas fracciones                   | Exacta para enteros                              |
| Rango útil  | Enteros seguros entre $\pm(2^{53} - 1)$             | Magnitud arbitraria, limitada por memoria        |
| Objeto Math | Compatible                                          | No compatible                                    |
| Uso típico  | Medidas, porcentajes, interfaz y cálculos generales | Identificadores o contadores enteros muy grandes |

### 1.1 Cómo representa Number los valores

`Number` sigue el formato IEEE 754 binario de 64 bits. De manera conceptual, un valor se compone de un bit de signo, 11 bits para el exponente y 52 bits almacenados para la fracción. Esto ofrece alrededor de 15 a 17 dígitos decimales significativos, pero no permite representar exactamente todas las fracciones decimales.

| Componente | Bits | Función                                                                                              |
|------------|------|------------------------------------------------------------------------------------------------------|
| Signo      | 1    | Indica si el valor es positivo o negativo.                                                           |
| Exponente  | 11   | Escala la magnitud mediante potencias de 2.                                                          |
| Fracción   | 52   | Almacena los dígitos significativos binarios; existe un bit principal implícito en valores normales. |

**Idea central:** Un decimal fácil de escribir en base 10 puede requerir infinitos dígitos en base 2. El motor almacena la aproximación binaria representable más cercana.

#### Código 1. Experimento de precisión binaria

```
const suma = 0.1 + 0.2;

console.log(suma);           // 0.30000000000000004
console.log(suma === 0.3);   // false
console.log(Math.abs(suma - 0.3)); // 5.551115123125783e-17
```

1. Abre las herramientas de desarrollo con F12 o Ctrl + Shift + I.
2. Selecciona la pestaña Console y ejecuta cada línea.
3. Describe la diferencia entre el valor matemático esperado y la aproximación almacenada.

## 1.2 Number.EPSILON no es una solución universal

Number.EPSILON es la distancia entre 1 y el siguiente Number representable mayor que 1. Puede ayudar a comparar resultados cercanos a esa escala, pero la tolerancia adecuada depende de la magnitud y del dominio. Para dinero, una estrategia didáctica más segura es trabajar con enteros en la unidad mínima: céntimos.

#### Código 2. Comparación con tolerancia relativa

```
function sonCasiIguales(a, b) {
  const escala = Math.max(1, Math.abs(a), Math.abs(b));
  return Math.abs(a - b) <= Number.EPSILON * escala;
}

console.log(sonCasiIguales(0.1 + 0.2, 0.3)); // true
```

**Advertencia:** No reemplaces una regla de negocio por una comparación aproximada sin definir la tolerancia. En sistemas financieros reales se emplean unidades enteras o bibliotecas decimales y reglas contables explícitas.

## 1.3 Valores especiales y comprobaciones

| Elemento             | Qué indica                                                                    | Comprobación recomendada    |
|----------------------|-------------------------------------------------------------------------------|-----------------------------|
| NaN                  | La operación no produjo un número válido.                                     | Number.isNaN(valor)         |
| Infinity / -Infinity | El resultado excede el rango o hubo una división entre cero no indeterminada. | Number.isFinite(valor)      |
| Entero               | No contiene parte fraccionaria.                                               | Number.isInteger(valor)     |
| Entero seguro        | Puede representarse y compararse sin ambigüedad.                              | Number.isSafeInteger(valor) |

#### Código 3. Verificación explícita de números

```
console.log(Number.isNaN(Number("abc"))); // true
console.log(Number.isFinite(10 / 0)); // false
console.log(Number.isInteger(12)); // true
console.log(Number.isSafeInteger(9007199254740991)); // true
console.log(Number.isSafeInteger(9007199254740992)); // false
```

**Recuerda:** `toFixed(2)` produce una cadena para presentación. No cambia el valor interno ni convierte una operación imprecisa en exacta.

## 1.4 Enteros seguros y BigInt

`Number.MAX_SAFE_INTEGER` equivale a  $2^{53} - 1$ . Más allá de ese límite, dos enteros matemáticamente distintos pueden terminar representados por el mismo `Number`. `BigInt` conserva enteros grandes de forma exacta.

### Código 4. Límite de Number y creación de BigInt

```
const limite = Number.MAX_SAFE_INTEGER;
console.log(limite); // 9007199254740991
console.log(limite + 1 === limite + 2); // true: se perdió distinción

const idExacto = 9007199254740993n;
const otroId = BigInt("9007199254740994");
console.log(idExacto + 1n === otroId); // true
```

| Regla de BigInt                                                    | Ejemplo                                                    |
|--------------------------------------------------------------------|------------------------------------------------------------|
| Usa el sufijo n o <code>BigInt()</code>                            | <code>123n</code> ; <code>BigInt("123")</code>             |
| No representa decimales                                            | <code>1.5n</code> produce <code>SyntaxError</code>         |
| La división entera trunca                                          | <code>7n / 2n</code> devuelve <code>3n</code>              |
| No mezcles <code>Number</code> y <code>BigInt</code> en aritmética | <code>1n + 1</code> produce <code>TypeError</code>         |
| Convierte de forma explícita y consciente                          | <code>Number(10n)</code> ; <code>BigInt(10)</code>         |
| <code>Math</code> no acepta <code>BigInt</code>                    | <code>Math.sqrt(16n)</code> produce <code>TypeError</code> |

**Uso en el proyecto:** `BigInt` generará un identificador didáctico de operación. No se utilizará para precios porque `BigInt` no representa fracciones; el dinero se manejará como `Number` entero en céntimos.

## 2. Expresiones, operadores y precedencia

Una expresión es una unidad de código que produce un valor. Un operador indica la operación y los operandos son los valores sobre los que actúa. La precedencia determina qué operación se evalúa primero; los paréntesis hacen explícita la intención y mejoran la lectura.

### Código 5. La precedencia cambia el resultado

```
const sinParentesis = 10 + 5 * 2; // 20
const conParentesis = (10 + 5) * 2; // 30
const potencia = 2 ** 3 ** 2; // 512: 2 ** (3 ** 2)

console.log({ sinParentesis, conParentesis, potencia });
```

**Buena práctica:** Usa paréntesis cuando la intención no sea evidente, aunque conozcas la precedencia.

## 2.1 Operadores aritméticos

| Operador | Nombre                  | Ejemplo    | Resultado              |
|----------|-------------------------|------------|------------------------|
| +        | Suma                    | 8 + 3      | 11                     |
| -        | Resta                   | 8 - 3      | 5                      |
| *        | Multiplicación          | 8 * 3      | 24                     |
| /        | División                | 8 / 3      | 2.666...               |
| %        | Resto                   | 8 % 3      | 2                      |
| **       | Exponenciación          | 2 ** 4     | 16                     |
| ++ / --  | Incremento / decremento | contador++ | Modifica en una unidad |
| +x / -x  | Signo unario            | +"12"; -5  | 12; -5                 |

**Cuidado con +:** Si uno de sus operandos es una cadena, + puede concatenar: 10 + "5" produce "105". Convierte la entrada con Number() y valida el resultado antes de operar.

## 2.2 Comparación y igualdad

| Operador | Propósito                       | Ejemplo                   |
|----------|---------------------------------|---------------------------|
| ===      | Igualdad estricta: valor y tipo | 5 === "5" → false         |
| !==      | Desigualdad estricta            | 5 !== "5" → true          |
| <, <=    | Menor que / menor o igual       | descuento <= 50           |
| >, >=    | Mayor que / mayor o igual       | cantidad >= 1             |
| ==, !=   | Igualdad con coerción           | Evitar como regla general |

### Código 6. Conversión explícita e igualdad estricta

```
const textoCantidad = "3";
const cantidad = Number(textoCantidad);

console.log(textoCantidad === 3); // false
console.log(cantidad === 3);      // true
console.log(Number.isInteger(cantidad) && cantidad > 0); // true
```

## 2.3 Operadores lógicos, cortocircuito y valor nulo

| Operador | Se interpreta como                                       | Aplicación                                           |
|----------|----------------------------------------------------------|------------------------------------------------------|
| &&       | Y lógico; devuelve el primer falsy o el último operando  | Validar varias condiciones                           |
|          | O lógico; devuelve el primer truthy o el último operando | Valor alternativo cuando cualquier falsy es inválido |
| !        | Negación lógica                                          | Invertir una condición                               |

| Operador          | Se interpreta como   | Aplicación                                  |
|-------------------|----------------------|---------------------------------------------|
| ??                | Coalescencia nula    | Usar alternativa solo ante null o undefined |
| condición ? a : b | Operador condicional | Elegir una de dos expresiones               |

### Código 7. Diferencia entre || y ??

```
const descuentoValido = 0;

console.log(descuentoValido || 10); // 10: 0 es falsy
console.log(descuentoValido ?? 10); // 0: no es null ni undefined

const envio = true;
const costo = envio ? 1500 : 0; // céntimos
```

**Falsy no significa ausente:** 0, cadena vacía, false, NaN, null y undefined son falsy. En cambio, ?? solo sustituye null y undefined. Elige el operador según la regla de negocio.

## 2.4 Operadores de asignación

| Forma   | Equivalencia                      | Uso                              |
|---------|-----------------------------------|----------------------------------|
| x += y  | x = x + y                         | Acumular                         |
| x -= y  | x = x - y                         | Descontar                        |
| x *= y  | x = x * y                         | Escalar                          |
| x /= y  | x = x / y                         | Dividir y reasignar              |
| x   = y | Asigna y si x es falsy            | Alternativa para cualquier falsy |
| x ??= y | Asigna y si x es null o undefined | Valor predeterminado nulo        |
| x &&= y | Asigna y si x es truthy           | Actualizar bajo condición truthy |

Las asignaciones compuestas son útiles cuando el cambio es evidente. Si la expresión se vuelve difícil de leer o tiene efectos secundarios, prefiere una instrucción más explícita.

## 2.5 Operadores de bits y desplazamiento

Con operandos Number, los operadores bit a bit convierten los valores a enteros de 32 bits. Son apropiados para máscaras, protocolos o banderas compactas; no deben usarse como truco para redondear ni para optimizar aritmética ordinaria.

| Operador | Nombre | Efecto                                                    |
|----------|--------|-----------------------------------------------------------|
| &        | AND    | Mantiene un bit cuando está activo en ambos operandos.    |
|          | OR     | Activa un bit cuando está activo en al menos un operando. |
| ^        | XOR    | Activa un bit cuando los operandos difieren.              |
| ~        | NOT    | Invierte los 32 bits.                                     |

| Operador | Nombre                           | Efecto                                         |
|----------|----------------------------------|------------------------------------------------|
| <<       | Desplazamiento izquierdo         | Mueve bits a la izquierda y agrega ceros.      |
| >>       | Desplazamiento derecho con signo | Replica el bit de signo.                       |
| >>>      | Desplazamiento derecho sin signo | Agrega ceros a la izquierda; no admite BigInt. |

### Código 8. Mini práctica: permisos como banderas

```
const VER = 1 << 0;      // 001
const EDITAR = 1 << 1;   // 010
const APROBAR = 1 << 2;  // 100

let permisos = 0;
permisos |= VER;
permisos |= EDITAR;      // 011

console.log((permisos & VER) !== 0);    // true
console.log((permisos & APROBAR) !== 0); // false

permisos &= ~EDITAR;      // elimina EDITAR: 001
console.log(permisos.toString(2).padStart(3, "0"));
```

1. Predice el valor binario después de activar VER y APROBAR.
2. Comprueba la predicción en la consola.
3. Explica por qué & sirve para consultar una bandera y | para activarla.

## 2.6 La misma escritura ..., dos funciones distintas

La sintaxis de tres puntos se interpreta según el contexto. Spread expande valores; rest reúne los argumentos restantes. En ambos casos la lectura de la expresión permite saber qué función cumple.

| Sintaxis           | Acción                                   | Ejemplo                              |
|--------------------|------------------------------------------|--------------------------------------|
| Spread en arreglo  | Expande elementos                        | const copia = [...original]          |
| Spread en objeto   | Copia propiedades enumerables propias    | const config = {...base, ...ajustes} |
| Spread en llamada  | Envía elementos como argumentos          | Math.max(...valores)                 |
| Rest en parámetros | Reúne argumentos restantes en un arreglo | function sumar(...valores)           |
| Rest en asignación | Reúne propiedades o elementos restantes  | const {id, ...datos} = objeto        |

**Código 9. Spread y rest en una misma operación**

```
const reglasBase = { igv: 18, envio: 0 };
const ajustes = { envio: 1500 };
const reglas = { ...reglasBase, ...ajustes };

function sumar(...valores) {
  let total = 0;
  for (const valor of valores) total += valor;
  return total;
}

const componentes = [10000, 1800, reglas.envio];
console.log(sumar(...componentes)); // 13300
```

**Copia superficial:** El spread de objetos y arreglos no clona recursivamente objetos anidados. Si una propiedad contiene otro objeto, ambas copias pueden seguir compartiendo esa referencia.

### 3. El objeto Math

Math agrupa propiedades y métodos estáticos para cálculos matemáticos. No se crea con new y sus métodos trabajan con Number, no con BigInt.

| Elemento            | Propósito                                | Ejemplo                |
|---------------------|------------------------------------------|------------------------|
| Math.round(x)       | Redondea al entero más cercano.          | Math.round(4.6) → 5    |
| Math.floor(x)       | Redondea hacia -Infinity.                | Math.floor(-1.2) → -2  |
| Math.ceil(x)        | Redondea hacia +Infinity.                | Math.ceil(1.2) → 2     |
| Math.trunc(x)       | Elimina la parte fraccionaria.           | Math.trunc(-1.9) → -1  |
| Math.abs(x)         | Obtiene el valor absoluto.               | Math.abs(-8) → 8       |
| Math.min / Math.max | Seleccionan el menor o el mayor.         | Math.max(2, 9, 4) → 9  |
| Math.sqrt(x)        | Calcula la raíz cuadrada.                | Math.sqrt(81) → 9      |
| Math.hypot(...x)    | Calcula la raíz de la suma de cuadrados. | Math.hypot(3, 4) → 5   |
| Math.random()       | Devuelve un pseudoaleatorio en [0, 1).   | No apto para seguridad |
| Math.PI             | Constante $\pi$ .                        | 2 * Math.PI * radio    |

**Código 10. Redondeo y número entero aleatorio**

```
const monto = 125.678;
console.log(Math.round(monto * 100) / 100); // 125.68

function enteroAleatorio(minimo, maximo) {
  return Math.floor(Math.random() * (maximo - minimo + 1)) + minimo;
}

console.log(enteroAleatorio(1, 6)); // entero entre 1 y 6
```

**Seguridad:** Math.random() es adecuado para simulaciones o interfaces, pero no para contraseñas, tokens, sorteos auditable ni decisiones criptográficas. Para seguridad se requieren APIs criptográficas y un diseño especializado.

### 3.1 Experimento de redondeo con valores negativos

#### Código 11. floor, ceil, trunc y round no son equivalentes

```
const valor = -2.7;

console.log(Math.floor(valor)); // -3
console.log(Math.ceil(valor)); // -2
console.log(Math.trunc(valor)); // -2
console.log(Math.round(valor)); // -3
```

1. Dibuja mentalmente -2.7 en una recta numérica.
2. Relaciona floor con la dirección hacia -Infinity y ceil con +Infinity.
3. Explica por qué trunc solo elimina la fracción y no significa siempre redondear hacia abajo.

## 4. Proyecto guiado: Cotizador comercial inteligente

El cotizador recibe un producto, precio, cantidad, descuento y dos opciones. Internamente convierte el precio a céntimos, calcula cada componente, crea un identificador BigInt y presenta importes formateados en soles.

### 4.1 Reglas funcionales

- El precio debe ser mayor que S/ 0.00, admitir como máximo dos decimales y no superar S/ 1 000 000.00.
- La cantidad debe ser un entero entre 1 y 10 000.
- El descuento solicitado debe ser un entero entre 0 % y 50 %.
- Un cliente frecuente recibe al menos 5 % de descuento; si solicita un porcentaje mayor, se conserva el mayor.
- El IGV es 18 % de la base imponible después del descuento.
- La entrega express agrega S/ 15.00 al total.
- Todos los importes se operan en céntimos enteros y se redondea cada monto derivado a un céntimo.

### 4.2 Fórmulas

| Concepto       | Fórmula en céntimos                                         |
|----------------|-------------------------------------------------------------|
| Subtotal       | precioUnitarioCentimos × cantidad                           |
| Descuento      | round(subtotal × porcentaje / 100)                          |
| Base imponible | subtotal - descuento                                        |
| IGV            | round(baselImponible × 18 / 100)                            |
| Envío          | 1500 si la bandera express está activa; 0 en caso contrario |
| Total          | baselImponible + IGV + envío                                |

### 4.3 Trazado manual antes de programar

Usa este caso de control: precio S/ 250.00, cantidad 1, descuento 0 %, cliente frecuente y envío express.

| Paso | Operación                             | Resultado                         |
|------|---------------------------------------|-----------------------------------|
| 1    | $25000 \times 1$                      | Subtotal: 25000 céntimos          |
| 2    | $\max(0 \%, 5 \%)$                    | Descuento aplicado: 5 %           |
| 3    | $\text{round}(25000 \times 5 / 100)$  | Descuento: 1250 céntimos          |
| 4    | $25000 - 1250$                        | Base: 23750 céntimos              |
| 5    | $\text{round}(23750 \times 18 / 100)$ | IGV: 4275 céntimos                |
| 6    | $23750 + 4275 + 1500$                 | Total: 29525 céntimos = S/ 295.25 |

**Criterio de aceptación:** Cuando el programa reciba ese caso, el total debe ser exactamente S/ 295.25.

### 4.4 Estructura del proyecto

1. Crea en tu carpeta de trabajo un directorio llamado cotizador-semana2.
2. Dentro crea el archivo index.html y las carpetas css y js.
3. En css crea styles.css; en js crea app.js.
4. Abre la carpeta completa desde Visual Studio Code mediante Archivo > Abrir carpeta.

| Ruta                             | Responsabilidad                                                   |
|----------------------------------|-------------------------------------------------------------------|
| cotizador-semana2/index.html     | Estructura, contenido, formulario y zonas de salida.              |
| cotizador-semana2/css/styles.css | Presentación, distribución, estados y adaptación móvil.           |
| cotizador-semana2/js/app.js      | Validación, reglas numéricas, operadores, cálculo, eventos y DOM. |

## 5. Construcción del archivo HTML

HTML define la estructura semántica. Los atributos min, max y step comunican restricciones al navegador, pero JavaScript volverá a validarlas porque el HTML no sustituye las reglas de negocio.

**Código 12. index.html - parte 1 de 5**

```

<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Cotizador didáctico para practicar números y operadores en
JavaScript.">
  <title>Cotizador comercial inteligente</title>
  <link rel="stylesheet" href="/css/styles.css">
  <script src="/js/app.js" defer></script>
</head>
<body>
  <header class="encabezado">
    <div class="contenedor">
      <p class="etiqueta">JavaScript Avanzado · Laboratorio 02</p>
      <h1>Cotizador comercial inteligente</h1>
      <p class="bajada">
        Calcula importes monetarios con operadores, enteros en céntimos y reglas explícitas.
      </p>
    </div>
  </header>

  <main class="contenedor cuadrícula">
    <section class="tarjeta" aria-labelledby="titulo-formulario">
      <p class="etiqueta">Datos de entrada</p>
      <h2 id="titulo-formulario">Configura la cotización</h2>

```

**Código 13. index.html - parte 2 de 5**

```

<form id="formCotizacion" novalidate>
  <div class="campo campo--completo">
    <label for="producto">Producto o servicio</label>
    <input id="producto" name="producto" type="text" maxlength="60"
      placeholder="Ej.: Monitor profesional" required>
  </div>

  <div class="campo">
    <label for="precio">Precio unitario (S/)</label>
    <input id="precio" name="precio" type="number" min="0.01"
      max="1000000" step="0.01" inputmode="decimal"
      placeholder="0.00" required>
  </div>

  <div class="campo">
    <label for="cantidad">Cantidad</label>
    <input id="cantidad" name="cantidad" type="number" min="1"
      max="10000" step="1" value="1" required>
  </div>

  <div class="campo campo--completo">
    <label for="descuento">Descuento solicitado (%)</label>
    <input id="descuento" name="descuento" type="number" min="0"
      max="50" step="1" value="0" required>
    <small>El descuento máximo permitido es 50 %.</small>
  </div>

```

**Código 14. index.html - parte 3 de 5**

```

    <fieldset class="opciones campo--completo">
      <legend>Opciones comerciales</legend>
      <label class="opcion">
        <input id="clienteFrecuente" type="checkbox">
        Cliente frecuente (mínimo 5 % de descuento)
      </label>
      <label class="opcion">
        <input id="envioExpres" type="checkbox">
        Entrega express (S/ 15.00)
      </label>
    </fieldset>

    <p id="mensajeError" class="mensaje-error" role="alert" hidden></p>

    <div class="acciones campo--completo">
      <button type="submit">Calcular cotización</button>
      <button type="reset" class="boton-secundario">Limpiar</button>
    </div>
  </form>
</section>

<section id="panelResultado" class="tarjeta resultado"
  aria-labelledby="titulo-resultado" aria-live="polite" hidden>
  <p class="etiqueta">Resultado</p>
  <h2 id="titulo-resultado">Resumen de la cotización</h2>

```

**Código 15. index.html - parte 4 de 5**

```

  <p class="operacion">
    Operación <output id="idOperacion"></output>
  </p>
  <p id="productoResultado" class="producto-resultado"></p>

  <dl class="resumen">
    <div><dt>Subtotal</dt><dd><output id="subtotalResultado"></output></dd></div>
    <div><dt>Descuento</dt><dd><output id="descuentoResultado"></output></dd></div>
    <div><dt>Base imponible</dt><dd><output id="baseResultado"></output></dd></div>
    <div><dt>IGV (18 %)</dt><dd><output id="igvResultado"></output></dd></div>
    <div><dt>Envío</dt><dd><output id="envioResultado"></output></dd></div>
    <div class="total"><dt>Total</dt><dd><output id="totalResultado"></output></dd></div>
  </dl>

  <p id="explicacionBanderas" class="detalle-banderas"></p>
</section>

<aside class="tarjeta nota campo--completo" aria-labelledby="titulo-precision">
  <h2 id="titulo-precision">¿Por qué se calcula en céntimos?</h2>
  <p>
    Javascript representa los valores <code>Number</code> con punto flotante binario.
    La aplicación convierte el precio a un entero de céntimos antes de operar y solo
    vuelve a soles al presentar el resultado.
  </p>
</aside>
</main>

```

**Código 16. index.html - parte 5 de 5**

```

<footer>
  <div class="contenedor">UTP · JavaScript Avanzado · Semana 2</div>
</footer>
</body>
</html>

```

## 5.1 Comprender la estructura

- **meta charset y viewport** aseguran codificación UTF-8 y adaptación a dispositivos móviles.
- **script defer** descarga JavaScript sin bloquear el análisis del HTML y lo ejecuta después de construir el DOM.
- **label for e input id** relacionan cada etiqueta con su control.
- **fieldset y legend** agrupan opciones relacionadas de forma semántica.
- **role=alert** anuncia errores y aria-live=polite informa cambios del resultado.
- **output** representa valores calculados y hidden controla la visibilidad inicial.

**Verificación inmediata:** Guarda y abre index.html en el navegador. Debes ver la estructura sin el diseño final. Si aparece una página vacía, revisa cierres de etiquetas y rutas relativas.

## 6. Presentación con CSS

CSS organiza el formulario y el resumen en dos columnas, conserva una jerarquía clara, comunica errores y adapta la interfaz a pantallas estrechas. Copia el contenido completo en css/styles.css.

### Código 17. styles.css - parte 1 de 12

```
:root {
  color-scheme: light;
  --azul-900: #0b2545;
  --azul-700: #1f4d78;
  --azul-500: #2e74b5;
  --azul-100: #e8f1fb;
  --rojo-500: #e63b2e;
  --gris-900: #111827;
  --gris-600: #5b6472;
  --gris-300: #cbd5e1;
  --gris-100: #f3f6fa;
  --blanco: #ffffff;
  --verde-700: #13795b;
  --sombra: 0 16px 36px rgb(11 37 69 / 10%);
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  color: var(--gris-900);
  background: #f7f9fc;
  font-family: Inter, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  line-height: 1.55;
```

**Código 18. styles.css - parte 2 de 12**

```
}

button,
input {
  font: inherit;
}

.contenedor {
  width: min(1100px, calc(100% - 2rem));
  margin-inline: auto;
}

.encabezado {
  padding: 3.5rem 0 6.5rem;
  color: var(--blanco);
  background:
    radial-gradient(circle at 85% 20%, rgb(255 255 255 / 16%), transparent 28%),
    linear-gradient(135deg, var(--azul-900), var(--azul-500));
}

.encabezado h1 {
  max-width: 760px;
  margin: 0.35rem 0 0.75rem;
  font-size: clamp(2rem, 5vw, 3.6rem);
  line-height: 1.05;
}
```

**Código 19. styles.css - parte 3 de 12**

```
.etiqueta {
  margin: 0;
  color: var(--azul-500);
  font-size: 0.78rem;
  font-weight: 800;
  letter-spacing: 0.08em;
  text-transform: uppercase;
}

.encabezado .etiqueta {
  color: #cfe6ff;
}

.bajada {
  max-width: 690px;
  margin: 0;
  color: #e6f2ff;
  font-size: 1.08rem;
}

.cuadrícula {
  display: grid;
  grid-template-columns: minmax(0, 1.08fr) minmax(320px, 0.92fr);
  gap: 1.5rem;
  margin-top: -4rem;
}
```

**Código 20. styles.css - parte 4 de 12**

```
padding-bottom: 3rem;
}

.tarjeta {
padding: clamp(1.25rem, 3vw, 2rem);
border: 1px solid #dfe7f0;
border-radius: 1rem;
background: var(--blanco);
box-shadow: var(--sombra);
}

.tarjeta h2 {
margin: 0.2rem 0 1.3rem;
color: var(--azul-900);
font-size: 1.45rem;
}

form {
display: grid;
grid-template-columns: repeat(2, minmax(0, 1fr));
gap: 1rem;
}

.campo {
display: grid;
gap: 0.35rem;
}
```

**Código 21. styles.css - parte 5 de 12**

```
}

.campo--completo {
grid-column: 1 / -1;
}

label,
legend {
color: var(--azul-900);
font-weight: 700;
}

input[type="text"],
input[type="number"] {
width: 100%;
min-height: 2.8rem;
padding: 0.65rem 0.75rem;
border: 1px solid var(--gris-300);
border-radius: 0.55rem;
color: var(--gris-900);
background: var(--blanco);
}

input:focus-visible,
button:focus-visible {
outline: 3px solid rgb(46 116 181 / 28%);
}
```

**Código 22. styles.css - parte 6 de 12**

```
    outline-offset: 2px;
  }

  small {
    color: var(--gris-600);
  }

  .opciones {
    display: grid;
    gap: 0.65rem;
    margin: 0;
    padding: 1rem;
    border: 1px solid var(--gris-300);
    border-radius: 0.65rem;
  }

  .opcion {
    display: flex;
    gap: 0.55rem;
    align-items: flex-start;
    font-weight: 500;
  }

  .opcion input {
    width: 1.1rem;
    height: 1.1rem;
  }
```

**Código 23. styles.css - parte 7 de 12**

```
    margin-top: 0.2rem;
    accent-color: var(--azul-500);
  }

  .acciones {
    display: flex;
    flex-wrap: wrap;
    gap: 0.75rem;
  }

  button {
    min-height: 2.8rem;
    padding: 0.7rem 1.05rem;
    border: 1px solid var(--azul-500);
    border-radius: 0.55rem;
    color: var(--blanco);
    background: var(--azul-500);
    font-weight: 800;
    cursor: pointer;
  }

  button:hover {
    background: var(--azul-700);
  }

  .boton-secundario {
```

**Código 24. styles.css - parte 8 de 12**

```
    color: var(--azul-700);
    background: var(--blanco);
}

.mensaje-error {
  grid-column: 1 / -1;
  margin: 0;
  padding: 0.8rem 0.9rem;
  border-left: 4px solid var(--rojo-500);
  border-radius: 0.25rem;
  color: #8f1d16;
  background: #fff0ee;
  font-weight: 700;
}

.resultado {
  align-self: start;
}

.operacion {
  color: var(--gris-600);
  font-size: 0.9rem;
}

.producto-resultado {
  margin: 0.5rem 0 1rem;
```

**Código 25. styles.css - parte 9 de 12**

```
    color: var(--azul-900);
    font-weight: 800;
}

.resumen {
  display: grid;
  gap: 0;
  margin: 0;
}

.resumen div {
  display: flex;
  justify-content: space-between;
  gap: 1rem;
  padding: 0.75rem 0;
  border-bottom: 1px solid #e4eaf1;
}

.resumen dt {
  color: var(--gris-600);
}

.resumen dd {
  margin: 0;
  color: var(--azul-900);
  font-variant-numeric: tabular-nums;
```

**Código 26. styles.css - parte 10 de 12**

```
    font-weight: 800;
  }

  .resumen .total {
    margin-top: 0.6rem;
    padding: 0.9rem;
    border: 0;
    border-radius: 0.65rem;
    background: var(--azul-100);
    font-size: 1.15rem;
  }

  .detalle-banderas {
    margin: 1rem 0 0;
    color: var(--verde-700);
    font-size: 0.92rem;
    font-weight: 700;
  }

  .nota {
    grid-column: 1 / -1;
    border-left: 5px solid var(--azul-500);
    box-shadow: none;
  }

  .nota h2 {
```

**Código 27. styles.css - parte 11 de 12**

```
    margin-bottom: 0.45rem;
  }

  .nota p {
    margin: 0;
  }

  code {
    padding: 0.1rem 0.3rem;
    border-radius: 0.25rem;
    color: var(--azul-900);
    background: var(--gris-100);
    font-family: Consolas, "Courier New", monospace;
  }

  footer {
    padding: 1.5rem 0;
    color: #d9e9f9;
    background: var(--azul-900);
    text-align: center;
  }

  [hidden] {
    display: none !important;
  }
```

**Código 28. styles.css - parte 12 de 12**

```
@media (max-width: 780px) {  
  .cuadrícula {  
    grid-template-columns: 1fr;  
  }  
  
  form {  
    grid-template-columns: 1fr;  
  }  
  
  .campo,  
  .campo--completo {  
    grid-column: 1;  
  }  
}
```

## 6.1 Decisiones de diseño

- **Las propiedades personalizadas** en `:root` centralizan colores y permiten mantener consistencia.
- **`minmax(0, 1fr)`** evita que el contenido fuerce columnas más anchas que el contenedor.
- **`focus-visible`** presenta un indicador de teclado visible sin alterar innecesariamente el clic del mouse.
- **`font-variant-numeric`**: `tabular-nums` alinea visualmente los importes.
- **La media query** cambia la cuadrícula a una sola columna cuando el ancho es menor o igual a 780 px.

**Comprueba:** Recarga la página, reduce el ancho de la ventana y confirma que formulario y resultado quedan en una sola columna sin desplazamiento horizontal.

## 7. Programación de la lógica JavaScript

El archivo se organiza por responsabilidades: configuración, referencias DOM, conversiones, banderas, cálculo puro, presentación y eventos. Copia cada parte en orden dentro de `js/app.js`.

**Código 29. app.js - parte 1 de 9**

```

"use strict";

const REGLAS_BASE = Object.freeze({
  igvPorcentaje: 18,
  descuentoClienteFrecuente: 5,
  descuentoMaximo: 50,
  envioExpresCentimos: 1500,
});

const OPCION_CLIENTE_FRECUEnte = 1 << 0;
const OPCION_ENVIO_EXPRES = 1 << 1;

const formulario = document.querySelector("#formCotizacion");
const inputProducto = document.querySelector("#producto");
const inputPrecio = document.querySelector("#precio");
const inputCantidad = document.querySelector("#cantidad");
const inputDescuento = document.querySelector("#descuento");
const inputClienteFrecuente = document.querySelector("#clienteFrecuente");
const inputEnvioExpres = document.querySelector("#envioExpres");
const mensajeError = document.querySelector("#mensajeError");
const panelResultado = document.querySelector("#panelResultado");

const salidas = {
  id: document.querySelector("#idOperacion"),
  producto: document.querySelector("#productoResultado"),
  subtotal: document.querySelector("#subtotalResultado"),

```

**Código 30. app.js - parte 2 de 9**

```

  descuento: document.querySelector("#descuentoResultado"),
  base: document.querySelector("#baseResultado"),
  igv: document.querySelector("#igvResultado"),
  envio: document.querySelector("#envioResultado"),
  total: document.querySelector("#totalResultado"),
  banderas: document.querySelector("#explicacionBanderas"),
};

const formateadorMoneda = new Intl.NumberFormat("es-PE", {
  style: "currency",
  currency: "PEN",
  minimumFractionDigits: 2,
});

let correlativo = 0n;

function crearIdOperacion() {
  correlativo += 1n;
  return BigInt(Date.now()) * 1_000_000n + correlativo;
}

function convertirImporteACentimos(texto) {
  const limpio = texto.trim();
  const partes = limpio.split(".");

  if (limpio === "" || partes.length > 2) {

```

**Código 31. app.js - parte 3 de 9**

```

    throw new TypeError("Ingresa un precio válido con punto decimal.");
  }

  const parteEntera = partes[0];
  const parteDecimal = partes[1] ?? "";

  if (parteEntera === "" || parteDecimal.length > 2) {
    throw new RangeError("El precio admite como máximo dos decimales.");
  }

  const enteros = Number(parteEntera);
  const decimales = Number(parteDecimal.padEnd(2, "0") || "0");

  if (
    !Number.isInteger(enteros) || enteros < 0 ||
    !Number.isInteger(decimales) || decimales < 0 || decimales > 99
  ) {
    throw new TypeError("El precio contiene caracteres o signos no válidos.");
  }

  const centimos = enteros * 100 + decimales;

  if (!Number.isSafeInteger(centimos) || centimos <= 0) {
    throw new RangeError("El precio debe ser mayor que 0 y estar dentro del rango permitido.");
  }
}

```

**Código 32. app.js - parte 4 de 9**

```

    return centimos;
  }

  function leerEntero(input, nombre, minimo, maximo) {
    const valor = Number(input.value);

    if (!Number.isInteger(valor) || valor < minimo || valor > maximo) {
      throw new RangeError(`${nombre} debe ser un entero entre ${minimo} y ${maximo}.`);
    }

    return valor;
  }

  function crearBanderas({ clienteFrecuente, envioExpres }) {
    let banderas = 0;

    if (clienteFrecuente) {
      banderas |= OPCION_CLIENTE_FRECUENTE;
    }

    if (envioExpres) {
      banderas |= OPCION_ENVIO_EXPRES;
    }

    return banderas;
  }
}

```

**Código 33. app.js - parte 5 de 9**

```
function tieneOpcion(banderas, opcion) {
  return (banderas & opcion) !== 0;
}

function sumarCentimos(...valores) {
  let total = 0;

  for (const valor of valores) {
    total += valor;
  }

  return total;
}

function calcularCotizacion(datos, opciones = {}) {
  const reglas = { ...REGLAS_BASE, ...opciones };
  const banderas = reglas.banderas ?? 0;
  const subtotalCentimos = datos.precioCentimos * datos.cantidad;

  if (!Number.isSafeInteger(subtotalCentimos)) {
    throw new RangeError("El subtotal excede el rango de enteros seguros.");
  }

  const esFrecuente = tieneOpcion(banderas, OPCION_CLIENTE_FRECUENTE);
  const descuentoMinimo = esFrecuente ? reglas.descuentoClienteFrecuente : 0;

```

**Código 34. app.js - parte 6 de 9**

```
const descuentoAplicado = Math.min(
  Math.max(datos.descuento, descuentoMinimo),
  reglas.descuentoMaximo,
);

const descuentoCentimos = Math.round(
  subtotalCentimos * descuentoAplicado / 100,
);
const baseImponibleCentimos = subtotalCentimos - descuentoCentimos;
const igvCentimos = Math.round(
  baseImponibleCentimos * reglas.igvPorcentaje / 100,
);
const envioCentimos = tieneOpcion(banderas, OPCION_ENVIO_EXPRES)
  ? reglas.envioExpresCentimos
  : 0;

const componentes = [baseImponibleCentimos, igvCentimos, envioCentimos];
const totalCentimos = sumarCentimos(...componentes);

return {
  ...datos,
  banderas,
  descuentoAplicado,
  subtotalCentimos,
  descuentoCentimos,
  baseImponibleCentimos,

```

**Código 35. app.js - parte 7 de 9**

```

    igvCentimos,
    envioCentimos,
    totalCentimos,
  };
}

function formatearCentimos(centimos) {
  return formateadorMoneda.format(centimos / 100);
}

function limpiarError() {
  mensajeError.textContent = "";
  mensajeError.hidden = true;
}

function mostrarError(mensaje) {
  mensajeError.textContent = mensaje;
  mensajeError.hidden = false;
}

function mostrarResultado(resultado) {
  salidas.id.value = crearIdOperacion().toString();
  salidas.producto.textContent = `${resultado.producto} x ${resultado.cantidad}`;
  salidas.subtotal.value = formatearCentimos(resultado.subtotalCentimos);
  salidas.descuento.value = `-${formatearCentimos(resultado.descuentoCentimos)}
  (${formatearCentimos(resultado.descuentoAplicado)} %)`;
  salidas.base.value = formatearCentimos(resultado.baseImponibleCentimos);

```

**Código 36. app.js - parte 8 de 9**

```

salidas.igv.value = formatearCentimos(resultado.igvCentimos);
salidas.envio.value = formatearCentimos(resultado.envioCentimos);
salidas.total.value = formatearCentimos(resultado.totalCentimos);

const frecuente = tieneOpcion(resultado.banderas, OPCION_CLIENTE_FRECUENTE);
const expres = tieneOpcion(resultado.banderas, OPCION_ENVIO_EXPRES);
salidas.banderas.textContent =
  `Banderas ${resultado.banderas.toString(2).padStart(2, "0")}: ` +
  `cliente frecuente ${frecuente ? "sí" : "no"}; ` +
  `envío express ${expres ? "sí" : "no"}.`;

panelResultado.hidden = false;
}

function manejarEnvio(evento) {
  evento.preventDefault();
  limpiarError();

  try {
    const producto = inputProducto.value.trim();

    if (producto === "") {
      throw new TypeError("Escribe el nombre del producto o servicio.");
    }

    const precioCentimos = convertirImporteACentimos(inputPrecio.value);

```

**Código 37. app.js - parte 9 de 9**

```

const cantidad = leerEntero(inputCantidad, "La cantidad", 1, 10000);
const descuento = leerEntero(inputDescuento, "El descuento", 0, 50);
const banderas = crearBanderas({
  clienteFrecuente: inputClienteFrecuente.checked,
  envioExpres: inputEnvioExpres.checked,
});

const datos = { producto, precioCentimos, cantidad, descuento };
const resultado = calcularCotizacion(datos, { banderas });
mostrarResultado(resultado);
} catch (error) {
  panelResultado.hidden = true;
  mostrarError(error instanceof Error ? error.message : "Ocurrió un error inesperado.");
}
}

function manejarReinicio() {
  limpiarError();
  panelResultado.hidden = true;
  queueMicrotask(() => inputProducto.focus());
}

formulario.addEventListener("submit", manejarEnvio);
formulario.addEventListener("reset", manejarReinicio);
inputProducto.focus();

```

**7.1 Lectura por responsabilidades**

| Bloque       | Elementos                          | Responsabilidad                                         |
|--------------|------------------------------------|---------------------------------------------------------|
| Reglas       | REGLAS_BASE                        | Centralizar porcentajes, límites y costo de envío.      |
| Bits         | $1 \ll 0, 1 \ll 1$                 | Asignar un bit independiente a cada opción.             |
| DOM          | querySelector()                    | Conservar referencias a entradas, mensajes y salidas.   |
| BigInt       | crearIdOperacion()                 | Generar un identificador entero grande de demostración. |
| Conversión   | convertirImporteACentimos()        | Convertir el texto decimal a un entero de céntimos.     |
| Validación   | leerEntero()                       | Rechazar cantidad o descuento fuera de la regla.        |
| Banderas     | crearBanderas(), tieneOpcion()     | Activar con OR y consultar con AND.                     |
| Rest/spread  | sumarCentimos(...), ...componentes | Reunir y expandir importes.                             |
| Cálculo      | calcularCotizacion()               | Aplicar reglas sin modificar el DOM.                    |
| Presentación | mostrarResultado()                 | Formatear y escribir el resultado accesible.            |
| Orquestación | manejarEnvio()                     | Leer, validar, calcular, mostrar y capturar errores.    |

**7.2 Por qué el precio se convierte desde texto**

El valor de un input llega como cadena. En lugar de operar primero con un decimal binario y después redondear, `convertirImporteACentimos()` separa la parte entera y la decimal, completa dos dígitos y construye un entero. Así, 19.90 se convierte en 1990 céntimos.

**Alcance didáctico:** La función acepta punto como separador decimal porque `input.value` usa una representación normalizada. Para producción internacional se requiere una estrategia de localización y validación más amplia.

### 7.3 Cómo se integran los operadores

| Concepto             | Uso dentro del programa                                          |
|----------------------|------------------------------------------------------------------|
| Aritmética           | Multiplica precio y cantidad; resta descuento; suma componentes. |
| Comparación          | Verifica rangos, enteros seguros y cadenas vacías.               |
| Lógicos              | Combina condiciones de validación y selecciona mensajes.         |
| Asignación compuesta | correlativo += 1n; banderas  = OPCION; total += valor.           |
| Coalescencia nula    | reglas.banderas ?? 0 conserva el valor 0 válido.                 |
| Condicional          | Elige descuento mínimo, envío y palabras sí/no.                  |
| Bits                 | Activa y consulta las opciones frecuente y express.              |
| Spread               | Combina reglas, expande componentes y construye el resultado.    |
| Rest                 | Reúne los componentes enviados a <code>sumarCentimos()</code> .  |
| Math                 | Limita porcentajes y redondea descuento e IGV.                   |

### 7.4 BigInt en el identificador

`Date.now()` produce un `Number` entero seguro que representa milisegundos. El programa lo convierte a `BigInt` antes de multiplicar y sumar el correlativo. El resultado solo se presenta como texto y nunca se mezcla con importes `Number`.

**Límite del ejemplo:** El identificador sirve para practicar `BigInt`; no reemplaza `UUID`, un identificador emitido por el servidor ni un mecanismo criptográficamente seguro.

## 8. Ejecución y verificación

1. Guarda `index.html`, `css/styles.css` y `js/app.js`.
2. Abre `index.html` con un navegador moderno. Este proyecto no usa módulos ni solicitudes remotas, por lo que puede abrirse como archivo local.
3. Abre DevTools, selecciona Console y confirma que no existan mensajes rojos.
4. Ejecuta la matriz de pruebas y compara cada salida con el valor esperado.

### 8.1 Matriz mínima de pruebas

| Caso | Entradas | Resultado esperado |
|------|----------|--------------------|
|------|----------|--------------------|

| Caso                        | Entradas                                | Resultado esperado                                            |
|-----------------------------|-----------------------------------------|---------------------------------------------------------------|
| 1. Básico                   | 100.10; cantidad 3; 0 %; sin opciones   | Subtotal S/ 300.30; IGV S/ 54.05; total S/ 354.35             |
| 2. Descuento                | 19.90; cantidad 2; 10 %; sin opciones   | Descuento S/ 3.98; base S/ 35.82; IGV S/ 6.45; total S/ 42.27 |
| 3. Frecuente + express      | 250.00; cantidad 1; 0 %; ambas opciones | Aplica 5 %; IGV S/ 42.75; envío S/ 15.00; total S/ 295.25     |
| 4. Precio vacío             | Sin precio                              | Mensaje: ingresa un precio válido                             |
| 5. Cantidad decimal         | Cantidad 1.5                            | Mensaje: debe ser un entero entre 1 y 10000                   |
| 6. Descuento fuera de rango | Descuento 51                            | Mensaje: debe ser un entero entre 0 y 50                      |

## 8.2 Verificación de banderas

| Frecuente | Express | Decimal | Binario mostrado |
|-----------|---------|---------|------------------|
| No        | No      | 0       | 00               |
| Sí        | No      | 1       | 01               |
| No        | Sí      | 2       | 10               |
| Sí        | Sí      | 3       | 11               |

Activa cada combinación, calcula y confirma que el mensaje del resultado coincida con la tabla.

## 8.3 Errores frecuentes y diagnóstico

| Síntoma                           | Causa probable                       | Cómo corregir                                                   |
|-----------------------------------|--------------------------------------|-----------------------------------------------------------------|
| No hay estilos                    | Ruta CSS incorrecta                  | Comprueba ./css/styles.css y el nombre exacto del archivo.      |
| formulario is null                | id distinto o script sin defer       | Revisa #formCotizacion y el atributo defer.                     |
| Cannot mix BigInt and other types | Se mezcló n con Number               | Convierte explícitamente o conserva un solo tipo por operación. |
| El total concatena texto          | Se operó con input.value             | Convierte y valida antes de usar +.                             |
| El descuento 0 se reemplaza       | Se usó    para un valor válido falsy | Usa ?? cuando solo null/undefined significan ausencia.          |
| El precio 1.999 se acepta         | Se omitió límite decimal             | Conserva la verificación parteDecimal.length > 2.               |
| El resultado no se actualiza      | Evento o selector incorrecto         | Coloca un breakpoint en manejarEnvio() y sigue el flujo.        |

## 8.4 Depuración guiada

1. Coloca un breakpoint en la primera línea de manejarEnvio().
2. Envía el formulario con el caso 3 y avanza con Step over.
3. Observa precioCentimos, cantidad, descuento y banderas.
4. Entra a calcularCotizacion() y verifica cada resultado intermedio.
5. Comprueba que totalCentimos sea 29525 antes de llamar a mostrarResultado().

**No depures a ciegas:** La consola y los breakpoints permiten verificar hipótesis; evita cambiar varias líneas sin identificar primero la causa.

## 9. Reto de extensión

Implementa al menos dos mejoras. Después de cada una, agrega un caso de prueba que demuestre su funcionamiento.

| Nivel | Mejora solicitada                                                                        | Concepto central           |
|-------|------------------------------------------------------------------------------------------|----------------------------|
| 1     | Envío gratuito cuando la base imponible sea al menos S/ 500.00 y no se elija express.    | && y operador condicional  |
| 1     | Mostrar cuánto falta para alcanzar el envío gratuito usando Math.max().                  | Math.max y resta           |
| 2     | Agregar una tercera bandera: instalación técnica por S/ 35.00.                           | 1 < 2,   y &               |
| 2     | Permitir un descuento predeterminado solo cuando el valor recibido sea null o undefined. | ?? o ??=                   |
| 3     | Crear una función calcularMedia(...valores) y probarla con spread.                       | rest, spread y acumulación |
| 3     | Mostrar una alerta si el subtotal deja de ser un entero seguro.                          | Number.isSafeInteger       |

## 10. Preguntas de reflexión

1. ¿Por qué toFixed(2) no resuelve por sí mismo los errores de precisión?
2. ¿Cuándo elegirías BigInt y por qué no lo usarías directamente para representar S/ 19.90?
3. ¿Qué diferencia funcional existe entre 0 || 10 y 0 ?? 10?
4. ¿Por qué (banderas & OPCION) !== 0 permite consultar una opción?
5. ¿Cómo distingues spread de rest si ambos se escriben con ...?
6. ¿Qué caso de prueba detectaría una mezcla accidental entre texto y número?

**Cierre:** El objetivo no es memorizar una lista de símbolos. Debes poder justificar qué tipo numérico y qué operador expresan con mayor claridad cada regla del problema.

## Fuentes de consulta

- [ECMAScript 2026 Language Specification](#)
- [MDN - Number](#)
- [MDN - Number.isSafeInteger\(\)](#)
- [MDN - BigInt](#)
- [MDN - Expressions and operators](#)
- [MDN - Bitwise operators](#)
- [MDN - Spread syntax](#)
- [MDN - Rest parameters](#)
- [MDN - Math](#)
- [MDN - Number.EPSILON](#)
- [MDN - Intl.NumberFormat](#)